

EDUCATIONAL ANALYSIS

Malware Architecture Analysis

Understanding How Malware Works Through Mirai and Zeus Source Code

Cybersecurity Education Series

June 2026

Table of Contents

1. Introduction to Malware Architecture

- 1.1 The Cyber Kill Chain
- 1.2 Core Malware Components

2. Case Study: Mirai Botnet

- 2.1 Architecture Overview
- 2.2 Infection Mechanism
- 2.3 Scanner Component
- 2.4 Command and Control
- 2.5 DDoS Attack Modules
- 2.6 Anti-Competition Mechanisms

3. Case Study: Zeus Banking Trojan

- 3.1 Architecture Overview
- 3.2 Infection and Deployment
- 3.3 Man-in-the-Browser Engine
- 3.4 Web Injects and Form Grabbing
- 3.5 Peer-to-Peer C2 Architecture
- 3.6 Domain Generation Algorithm

4. Malware Analysis Methodologies

- 4.1 Static Analysis
- 4.2 Dynamic Analysis
- 4.3 Network Traffic Analysis

5. Defensive Strategies

- 5.1 IoT Botnet Mitigation
- 5.2 Banking Trojan Defense

6. Conclusion

References

1. Introduction to Malware Architecture

Malware represents one of the most significant threats in modern cybersecurity, evolving from simple viruses to sophisticated, multi-component attack platforms. Understanding how malware works at the architectural level is essential for defenders, incident responders, and security researchers. This document provides a comprehensive technical analysis of two landmark malware families: **Mirai**, an IoT botnet responsible for some of the largest distributed denial-of-service (DDoS) attacks in history, and **Zeus** (Zbot), a banking trojan that established the foundational architecture for financial cybercrime.

Both malware families have had their source code leaked or publicly released, making them invaluable educational case studies. By analyzing their internal structures, infection mechanisms, command-and-control architectures, and defensive countermeasures, we can understand the principles that underpin modern malware operations.

Educational Purpose Notice

This document is intended solely for educational and defensive cybersecurity purposes. All analysis is based on publicly available research, leaked source code, and documented incident reports. Understanding malware architecture is critical for building effective defenses.

1.1 The Cyber Kill Chain

Malware operations follow a structured lifecycle modeled by the **cyber kill chain**, originally developed by Lockheed Martin. Understanding this framework helps defenders identify and interrupt attacks at each stage.

Table 1 Cyber Kill Chain Stages

Stage	Description	Defensive Opportunity
Reconnaissance	Target selection and information gathering	Threat intelligence, footprint reduction
Weaponization	Payload creation and packaging	Signature development, patch management
Delivery	Transmission via email, web, or physical means	Email filtering, web proxies, user training
Exploitation	Triggering vulnerabilities to execute code	Endpoint protection, vulnerability scanning
Installation	Establishing persistence and backdoors	Behavioral detection, integrity monitoring
Command and Control	Establishing communication with attacker	Network monitoring, DNS filtering, EDR
Actions on Objectives	Data theft, destruction, or lateral movement	DLP, segmentation, incident response

Mirai and Zeus both follow this kill chain but specialize in different stages. Mirai excels at automated exploitation and propagation at massive scale, while Zeus focuses on stealthy persistence and credential harvesting. Both, however, rely critically on command-and-control (C2) infrastructure to achieve their objectives.

1.2 Core Malware Components

Despite their different targets, both Mirai and Zeus share fundamental architectural components that are common to most sophisticated malware:

Infection Vector: The mechanism by which malware initially compromises a system. Mirai uses automated scanning and brute-force attacks against Telnet and SSH services. Zeus typically spreads through phishing emails, exploit kits, and drive-by downloads.

Persistence Mechanism: Techniques to survive system reboots and maintain long-term access. Mirai achieves persistence through process reinfection and cron jobs. Zeus employs registry modifications, hidden files, and API hooking to evade detection.

Command and Control (C2): The communication infrastructure that allows attackers to remotely control compromised systems. Mirai uses a centralized C2 model, while Zeus (in its GameOver variant) pioneered peer-to-peer C2 architectures.

Payload Module: The component that executes the malware's primary mission. For Mirai, this is the DDoS attack engine. For Zeus, it is the banking credential harvester with web injection capabilities.

2. Case Study: Mirai Botnet

Mirai (Japanese for "future") is a malware that targets consumer devices such as IP cameras and home routers, enslaving them into a botnet for large-scale DDoS attacks. First identified in August 2016, Mirai gained notoriety for the massive DDoS attack against Krebs on Security and the Dyn DNS provider, which disrupted access to major websites including Twitter, Netflix, and Reddit across Europe and North America.

The source code for Mirai was publicly released in October 2016, leading to numerous variants and a significant evolution in IoT botnet capabilities. Mirai's authors were eventually identified and prosecuted, but its code continues to spawn new variants including Satori, Wicked, OMG, and more recently, Jackskid.

2.1 Architecture Overview

Mirai consists of four primary components that work together to form a self-propagating botnet:

Table 2 Mirai Botnet Components

Component	Function	Location
CNC Server	Database of bots; receives heartbeats and issues attack commands	Attacker-controlled VPS
Scan Listener	Receives successful brute-force reports from bots	Dedicated server
Loader	Infects vulnerable devices by logging in and deploying malware	High-bandwidth servers
Bot (Malware)	Resides on infected devices; scans and brute-forces new targets	Compromised IoT devices

The architecture is modular by design. The CNC server maintains a database of all active bots. When a bot successfully brute-forces a new device, it reports the credentials to the Scan Listener, which forwards them to the Loader. The Loader then logs into the target device, determines its architecture (ARM, MIPS, x86, etc.), downloads the appropriate binary, and executes it.

2.2 Infection Mechanism

Mirai's infection process follows a systematic pipeline designed for maximum propagation speed:

Step 1: Scanning. Each infected bot continuously scans random IPv4 addresses for open Telnet (port 23) and SSH (port 2323) services. Mirai uses a custom pseudorandom number generator (PRNG) to select target IP addresses, deliberately avoiding certain ranges including loopback addresses (127.0.0.0/8), IANA reserved addresses, and networks belonging to the US Department of Defense, General Electric, and Hewlett-Packard.

Step 2: Brute-Force Authentication. When an open port is found, Mirai attempts to log in using a built-in dictionary of default and weak credentials. The credential list includes combinations such as admin/admin, root/root, support/support, and many IoT device-specific defaults.

Step 3: Reporting. Upon successful login, the bot reports the target IP address, port, and working credentials to the Scan Listener on port 48101.

Step 4: Loading. The Loader receives the report, authenticates to the target device, and executes commands to download and run the Mirai binary appropriate for the device's CPU architecture.

Step 5: Activation. The newly infected device becomes a bot, establishes a connection to the CNC server, and begins its own scanning operations, continuing the infection cycle.

Scanner Code Example

The following pseudocode illustrates Mirai's IP selection logic from scanner.c:

```
static ipv4_t get_random_ip(void) {
    uint32_t tmp;
    uint8_t o1, o2, o3, o4;
    do {
        tmp = rand_next();
        o1 = tmp & 0xff;
        o2 = (tmp >> 8) & 0xff;
        o3 = (tmp >> 16) & 0xff;
        o4 = (tmp >> 24) & 0xff;
    } while (
        o1 == 127 || // Loopback
        o1 == 0   || // Invalid
        o1 == 3   || // General Electric
        (o1 == 15 || o1 == 16) || // Hewlett-Packard
        // ... additional filters
    );
    return ...
}
```

2.3 Scanner Component

The scanner module (scanner.c) is one of Mirai's most sophisticated components. It implements an asynchronous SYN scanner that is approximately 80 times faster than previous botnet scanners while using 20 times fewer resources. The scanner uses raw TCP sockets to send SYN packets without completing the full three-way handshake, allowing rapid identification of open services.

Key technical features of the scanner include:

Raw Socket Operations: The scanner creates raw sockets to craft and send TCP packets directly, bypassing the operating system's TCP stack for maximum speed.

Asynchronous I/O: Instead of waiting for each connection to complete, the scanner maintains a large pool of pending connections and polls them for responses, enabling parallel scanning of thousands of hosts.

Architecture Detection: After gaining shell access, the loader uses commands like `cat /proc/cpuinfo` or `uname -m` to determine the target's CPU architecture and select the appropriate binary.

2.4 Command and Control

Mirai uses a centralized command-and-control architecture. Bots resolve a domain name (configured in `resolv.c/resolv.h`) to obtain the CNC server's IP address, then establish a persistent TCP connection. This connection serves two purposes: sending periodic heartbeat messages to maintain the bot's presence in the database, and receiving attack commands from the botmaster.

The CNC protocol uses a custom binary format. When the botmaster wants to launch an attack, they issue commands through the CNC server, which forwards them to all connected bots. Each command specifies the attack type, target IP address or domain, port number, and duration.

Mirai's C2 architecture, while simple, has proven effective. The centralized model makes the CNC server a single point of failure, but Mirai compensates by making the infrastructure easy to relocate and by using domain names rather than hardcoded IPs.

2.5 DDoS Attack Modules

Mirai includes multiple DDoS attack vectors, each implemented as a separate function in the attack module. Modern variants have expanded from the original 6 attack types to 16 or more. The attacks can be categorized as follows:

Table 3 Mirai DDoS Attack Vectors

Category	Attack Type	Description
UDP Floods	UDP Generic	High-volume UDP traffic to random ports
	UDP Plain	Simplified UDP flood without payload variation
	VSE Flood	Valve Source Engine query amplification
	DNS Flood	Targeted DNS query floods
TCP Attacks	SYN Flood	Raw socket SYN packets with IP spoofing
	ACK Flood	TCP ACK packet floods
	STOMP Attack	40-byte SYN packets with zero TCP options
	TCP Handshake	Completes 3-way handshake then floods PSH+ACK data
	TCP Bypass	Uses established connections to evade SYN cookies
Application-Layer	HTTP Flood	GET requests with Mozilla user-agent
	GRE Flood	GRE tunnel encapsulation to evade UDP filters
	Game Protocols	Minecraft, RakNet, A2S_INFO, FiveM floods

The TCP handshake flood (attack ID 11) is particularly notable for its evasion approach: it completes a legitimate three-way handshake via a STREAM socket, then sends PSH+ACK data floods via a separate RAW socket using captured sequence numbers. Because the attack uses established connections, it bypasses SYN cookie defenses and may evade stateful inspection devices.

2.6 Anti-Competition Mechanisms

Modern Mirai variants include aggressive anti-competition features designed to eliminate rival malware from infected devices. The Jackskid variant, for example, includes an "OclKiller" module that employs multiple methods to detect and terminate competing botnets:

Process Scanning: The killer module continuously scans `/proc` for processes matching known competitor signatures. It kills rivals by inspecting `/proc/<pid>/exe` paths, memory maps (`/proc/<pid>/maps`), and process statistics.

NETLINK Monitor: Some variants create an `AF_NETLINK` socket (proc connector) to receive real-time process creation events from the kernel, allowing them to kill competitor processes within milliseconds of spawning.

Port Blocking: Mirai uses iptables to block ports used by competing botnets, such as port 5555 (ADB) and port 3222, preventing reinfection by rivals.

Binary Disabling: Some variants bind-mount `/dev/null` over netcat and other utility binaries, preventing competing malware from using these tools for its own propagation.

3. Case Study: Zeus Banking Trojan

Zeus, also known as Zbot or TDSS, is a banking trojan that emerged in 2007 and fundamentally transformed the landscape of financial cybercrime. Unlike Mirai's focus on DDoS, Zeus was specifically engineered to steal banking credentials and financial information through man-in-the-browser (MitB) attacks. According to the U.S. Department of Justice, Zeus operations were responsible for stealing over \$100 million worldwide.

Zeus established the architectural blueprint that modern banking trojans continue to follow. After its source code leaked in 2011, it spawned numerous variants including Citadel, SpyEye, GameOver Zeus, and indirectly influenced Dridex and Trickbot. GameOver Zeus alone infected an estimated 3.6 million PCs in the United States before being disrupted by Operation Tovar in 2014.

3.1 Architecture Overview

Zeus employs a modular architecture designed for stealth, persistence, and adaptability:

Table 4 Zeus Trojan Core Components

Component	Function	Storage Location
Core Bot	Main executable, hooks APIs, manages modules	Windows system32 (hidden)
Configuration	Target bank list, web inject scripts, C2 addresses	local.ds (encrypted)
Keystroke Log	Captured keystrokes and form data	userds (encrypted)
Web Inject Module	Browser-specific DLLs for code injection	Injected into browser process
Hidden FS	Rootkit-hidden files evading detection	\windows\system32\lowsec\

The Zeus architecture pioneered several techniques that remain standard in banking malware today: form grabbing (capturing data before browser encryption), web injection (modifying legitimate banking pages in real-time), and a modular configuration system that allows operators to update targets without reinfecting systems.

3.2 Infection and Deployment

Zeus employs multiple infection vectors, with a strong reliance on social engineering:

Phishing Campaigns: The most common delivery method. Attackers send emails impersonating financial institutions, government agencies (FBI, IRS), or well-known companies. These emails contain malicious attachments or links that trigger the initial download.

Exploit Kits: Zeus operators deploy exploit kit infrastructure that profiles the victim's browser and delivers targeted exploits for unpatched vulnerabilities. When a user visits a compromised legitimate website, the exploit kit silently installs Zeus.

Drive-by Downloads: Compromised websites serve malware to visitors without requiring any user interaction beyond loading the page.

Once executed, Zeus follows a rapid deployment sequence:

1. **Dropper Execution:** The initial dropper unpacks the main Zeus binary into memory at a specific location (typically 0x00B70000) with PAGE_EXECUTE_READWRITE protection.
2. **Persistence Setup:** Zeus copies itself to the Windows system directory with a random name and modifies the registry Run key or creates a Windows service.

3. **API Hooking:** The malware injects itself into running processes (especially browsers) and hooks critical Windows APIs including NtQueryDirectoryFile (to hide files), LdrLoadDll, and various network functions.
4. **C2 Registration:** Zeus sends an HTTP GET request to its C2 server, downloads an encrypted configuration file (config.bin), and registers the infected system with an HTTP POST.
5. **Monitoring Activation:** With configuration loaded, Zeus begins monitoring browser activity for visits to targeted banking websites.

3.3 Man-in-the-Browser Engine

The man-in-the-browser (MitB) capability is Zeus's defining feature. Unlike simple keyloggers that record keystrokes, Zeus operates inside the browser process itself, intercepting and modifying web traffic at the application layer before SSL/TLS encryption occurs.

Technically, the MitB engine works through the following mechanism:

Process Injection: Zeus injects its DLL into browser processes (iexplore.exe, firefox.exe, chrome.exe) using standard Windows APIs like CreateRemoteThread and VirtualAllocEx. Once inside the browser process, it operates with the same privileges as the browser itself.

API Hooking: Within the browser process, Zeus hooks critical functions related to HTTP request handling, DOM manipulation, and form submission. This allows it to see all data entered into web forms before encryption.

SSL/TLS Bypass: Because Zeus intercepts data at the application layer (inside the browser, before the SSL library), it captures credentials and form data regardless of HTTPS encryption. The SSL certificate remains valid and green; the user has no visual indication that data is being stolen.

Session Hijacking: Beyond stealing credentials, Zeus can capture session cookies after successful multi-factor authentication, allowing attackers to hijack authenticated banking sessions even when passwords alone are insufficient.

3.4 Web Injects and Form Grabbing

Zeus's web injection system is its most sophisticated attack capability. The configuration file contains web inject rules that specify:

- **Target URLs:** Which banking websites to monitor

- **Injection Points:** Where in the page HTML to insert malicious content (using data_before/data_after matching)
- **Injection Content:** JavaScript or HTML to insert, such as additional form fields requesting PINs, social security numbers, or other sensitive data
- **ATS Scripts:** Automatic Transaction System scripts that perform wire transfers in the background without user interaction

When a user visits a targeted bank, Zeus's injected JavaScript can:

1. Add fake form fields that appear to be part of the legitimate banking interface
2. Modify account balance displays to hide fraudulent transactions
3. Automatically initiate wire transfers using captured credentials
4. Intercept and relay one-time passwords entered for two-factor authentication
5. Redirect transactions to attacker-controlled accounts while showing the user a fake confirmation

Web Injection Configuration Example

Zeus web inject configurations typically follow this pattern:

```
set_url https://online.bank.com/login* GP
data_before
<input type="password" name="passwd">
data_end
data_inject
<input type="text" name="ssn" placeholder="SSN (required)">
<input type="text" name="pin" placeholder="ATM PIN (required)">
data_end
```

This configuration tells Zeus to inject additional fields for SSN and PIN on the bank's login page, making users believe the bank requires this information.

3.5 Peer-to-Peer C2 Architecture

The GameOver Zeus variant introduced a revolutionary peer-to-peer (P2P) C2 architecture that eliminated the single point of failure inherent in Mirai's centralized model. This architecture was inspired by the Kademlia distributed hash table protocol and represented a significant advancement in botnet resilience.

The P2P architecture operates as follows:

Peer Network Formation: Each infected machine maintains a peer list of up to 150 contacts. When a new bot joins, it receives a subset of peers (typically 10) from existing

bots. Peer discovery uses XOR-based distance metrics similar to Kademlia.

Decentralized Command Distribution: Commands propagate through the network from peer to peer rather than from a central server. This means the botnet can function even if no single bot can reach the original C2 infrastructure.

Proxy Layer: Certain bots function as "supernodes" or proxy bots that relay data between the P2P network and attacker-controlled proxy servers. This adds an additional layer of obfuscation.

Update Propagation: Binary updates, configuration files, and proxy lists propagate laterally through the infected peer network. All updates are digitally signed with RSA-2048 to prevent unauthorized modifications.

Communication Protocol: GOZ uses UDP for lightweight control messages (peer exchanges) and TCP for bulk transfers (configuration updates, stolen data). Traffic is encrypted with RC4, with ports randomly selected in the 10,000-30,000 range.

3.6 Domain Generation Algorithm

GameOver Zeus incorporates a Domain Generation Algorithm (DGA) as a fallback mechanism when P2P connectivity fails. If a bot loses contact with all peers or cannot fetch updates for a week, it activates the DGA to generate potential C2 domains.

The DGA generates 1,000 unique domains per week using the following algorithm:

1. Take the MD5 hash of a concatenation containing the current year, month, week-of-month, and domain index (0-999)
2. Convert the MD5 hash into a domain name of up to 32 lowercase alphabetic characters
3. Select a top-level domain from the set {biz, com, info, net, org, ru} based on the index

The deterministic nature of this algorithm means both the attacker and the bots can calculate the same domain list for any given week. The attacker only needs to register a small number of the 1,000 possible domains, while defenders cannot practically block or sinkhole all of them.

The DGA proved to be a highly effective resilience mechanism. Even after the Operation Tovar takedown in June 2014, residual infections continued to regenerate C2 channels through the DGA, demonstrating the difficulty of permanently dismantling such architectures.

DGA Pseudocode

```

for (i = 0; i < 1000; i++) {
    // Create seed from date components
    data[0] = year + '0';
    data[1] = month;
    data[2] = 7 * (day / 7);
    *(DWORD*)(data+3) = i;

    // Generate MD5 hash
    md5 = MD5(data, 7);

    // Convert hash to domain name
    for (j = 0; j < 16; j++) {
        c1 = (md5[j] & 0x1F) + 'a';
        c2 = (md5[j] >> 3) + 'a';
        if (c1 != c2 && c1 <= 'z') name += c1;
        if (c1 != c2 && c2 <= 'z') name += c2;
    }

    // Select TLD based on index
    if (i % 6 == 0) name += ".ru";
    else if (i % 6 == 1) name += ".biz";
    else if (i % 6 == 2) name += ".info";
    else if (i % 6 == 3) name += ".org";
    else if (i % 6 == 4) name += ".net";
    else name += ".com";

    domains[i] = name;
}

```

4. Malware Analysis Methodologies

Understanding malware requires systematic analysis using multiple complementary approaches. Security researchers and incident responders employ three primary analysis methodologies:

4.1 Static Analysis

Static analysis examines malware without executing it, focusing on understanding the code structure, identifying capabilities, and extracting indicators of compromise (IOCs).

File Structure Analysis: Tools like Detect It Easy and PE-bear identify file type, packer/compiler signatures, section entropy, and import tables. Mirai binaries are typically ELF

files packed with UPX; Zeus samples are Windows PE files often packed with custom crypters.

Disassembly and Decompilation: IDA Pro, Ghidra, and Binary Ninja convert machine code to assembly or pseudocode. For Mirai (compiled from C), decompilation reveals the original logic structure. For Zeus (with anti-debugging), disassembly requires bypassing obfuscation first.

String Extraction: Tools like `strings` and FLOSS extract readable text from binaries including URLs, file paths, error messages, and configuration data. Mirai strings often reveal C2 domains and attack commands; Zeus strings may contain web inject signatures.

Signature and Hash Analysis: Comparing file hashes (MD5, SHA-256) against threat intelligence databases quickly identifies known malware families and variants.

4.2 Dynamic Analysis

Dynamic analysis executes malware in a controlled environment to observe its runtime behavior, network activity, and system modifications.

Sandbox Execution: Automated sandboxes (ANY.RUN, Cuckoo, Joe Sandbox) run malware in isolated virtual machines, recording all system calls, file operations, registry changes, and network connections.

Behavioral Monitoring: Tools like Procmon (Process Monitor) capture real-time file system, registry, and process/thread activity. For Zeus analysis, this reveals API hooking behavior and hidden file creation. For Mirai, it shows process creation and network scanning activity.

Memory Forensics: Volatility and Rekall analyze memory dumps to find injected code, unpacked payloads, network connections, and encryption keys that only exist in RAM. Zeus configurations and decrypted web injects are often recoverable only from memory.

Debugger Analysis: Debuggers (x64dbg, GDB, OllyDbg) allow step-by-step execution, breakpoint setting, and register/memory inspection. This is essential for unpacking obfuscated malware and understanding conditional behavior.

4.3 Network Traffic Analysis

Network analysis captures and examines the communication patterns between malware and its infrastructure.

Packet Capture: Wireshark and tcpdump record all network traffic from the analysis environment. For Mirai, this reveals scanning patterns, Telnet brute-force attempts, and C2

heartbeat traffic. For Zeus, it shows C2 registration, configuration downloads, and credential exfiltration.

DNS Monitoring: Since both Mirai and Zeus use DNS for C2 resolution, monitoring DNS queries is critical. Zeus's DGA produces distinctive patterns of high-entropy domain lookups that can be detected through DNS analytics.

TLS/SSL Inspection: Man-in-the-middle proxies (Burp Suite, mitmproxy) can decrypt HTTPS traffic when malware does not implement certificate pinning. This reveals the content of C2 communications including attack commands and stolen data.

Flow Analysis: NetFlow and Zeek extract connection metadata (source/destination IPs, ports, bytes transferred, duration) without capturing full packet content. This enables detection of beaconing patterns and anomalous outbound connections at scale.

5. Defensive Strategies

Effective defense against malware requires layered security measures addressing each stage of the kill chain. The following strategies are tailored to the specific threats posed by IoT botnets and banking trojans.

5.1 IoT Botnet Mitigation

Defending against Mirai-class botnets requires securing the devices that serve as infection targets:

Device Hardening: Change default passwords on all IoT devices before deployment. Disable Telnet and SSH if not required. Keep firmware updated with the latest security patches. These measures address Mirai's primary infection vectors directly.

Network Segmentation: Isolate IoT devices on separate network segments or VLANs with restricted outbound access. Mirai requires outbound connectivity to report to the Scan Listener and CNC server; limiting this access prevents infection confirmation and C2 communication.

Ingress/Egress Filtering: Block inbound connections to Telnet (23) and SSH (2323) ports from the internet. Filter outbound connections to known malicious IPs and domains using threat intelligence feeds. Rate-limit SYN packets to prevent scanning from compromised internal devices.

DDoS Protection: Deploy upstream DDoS mitigation services (Cloudflare, Akamai, AWS Shield) that can absorb and distribute large volumetric attacks. These services use anycast networks and traffic scrubbing centers to filter malicious traffic before it reaches the target.

Network Monitoring: Monitor for IoT botnet indicators including Telnet brute-force attempts, connections to known CNC IPs, SYN scan patterns, and unusual UDP/TCP flood traffic from internal devices. DNS query volume anomalies may indicate DGA activity from emerging variants.

5.2 Banking Trojan Defense

Defending against Zeus-class banking trojans requires endpoint security, network monitoring, and transaction verification:

Endpoint Detection and Response (EDR): Deploy EDR solutions that monitor for process injection, API hooking, and browser manipulation. Zeus's hallmark behaviors (injecting into browser processes, hooking NtQueryDirectoryFile) are detectable through behavioral analysis even when signature-based detection fails.

Browser Isolation: Remote browser isolation (RBI) solutions execute web browsing in a sandboxed cloud environment, preventing malware on the endpoint from accessing or manipulating banking pages. Since Zeus cannot inject into remote browser sessions, this effectively blocks MitB attacks.

Multi-Factor Authentication: Implement strong MFA for banking access, preferably using hardware security keys or out-of-band authentication. While Zeus can steal OTPs entered on the same device, it cannot compromise physically separate authentication channels.

Transaction Verification: Use out-of-band transaction confirmation (SMS, email, phone call) that includes transaction details (amount, destination account). Since Zeus cannot intercept communications on a separate device, this allows victims to detect and prevent fraudulent transfers.

Certificate Pinning: Implement certificate pinning in banking applications to detect MitM proxy attempts. While Zeus operates at the application layer (before SSL), pinning can detect some interception tools used alongside Zeus.

User Education: Train users to recognize phishing emails, avoid suspicious attachments, and verify banking website URLs. Since Zeus's initial infection often depends on social engineering, educated users represent a critical first line of defense.

Fraud Detection Systems: Server-side behavioral analytics monitor for anomalous banking patterns (unusual login times, locations, transaction amounts, or device fingerprints). These systems can detect Zeus activity even when the endpoint is fully compromised.

6. Conclusion

Mirai and Zeus represent two distinct but equally influential branches of malware evolution. Mirai demonstrated the devastating potential of IoT insecurity, weaponizing billions of poorly secured devices into a DDoS infrastructure capable of disrupting internet-scale services. Its centralized C2 architecture, rapid scanner, and modular attack toolkit established the template for IoT botnets that continue to proliferate today.

Zeus transformed financial cybercrime by pioneering man-in-the-browser attacks that bypass SSL encryption and steal credentials from within the browser itself. Its modular configuration system, P2P C2 architecture in GameOver Zeus, and domain generation algorithm set standards for resilience that modern banking trojans continue to emulate.

Both malware families share fundamental principles: modular design for adaptability, persistence mechanisms for longevity, and C2 infrastructure for remote control and data exfiltration. Understanding these principles is essential for effective defense. By analyzing leaked source code and reverse-engineering samples, security researchers can develop detection signatures, behavioral indicators, and defensive countermeasures that protect against both current threats and future variants.

The arms race between malware authors and defenders continues. Mirai variants now include anti-competition modules, encrypted configurations, and multi-architecture support. Zeus derivatives have evolved into multi-functional malware platforms combining banking fraud with ransomware deployment. The educational analysis of these landmark malware families provides the foundation for understanding and countering the next generation of cyber threats.

References

1. Kolias, C., Kambourakis, G., Stavrou, A., & Voas, J. (2017). DDoS in the IoT: Mirai and Other Botnets. *Computer*, 50(7), 80-84.
2. US Department of Justice. (2017). Justice Department Announces Charges And Guilty Pleas In Three Computer Crime Cases Involving Significant DDoS Attacks. Press Release.
3. Andriesse, D., Rossow, C., Stone-Gross, B., Plohmann, D., & Bos, H. (2015). Highly Resilient Peer-to-Peer Botnets Are Here: An Analysis of Gameover Zeus. In *2015 8th International Conference on Malicious and Unwanted Software (MALWARE)*, 116-125.

4. Caballero, J., Grier, C., Kreibich, C., & Paxson, V. (2011). Measuring Pay-per-Install: The Commoditization of Malware Distribution. In *USENIX Security Symposium*, 13-13.
5. Binks, R. (2011). Zeus / SpyEye - The Cybercrime Kit War. *Virus Bulletin Conference*.
6. Lelli, A. (2016). Mirai (DDoS) Source Code Review. Technical Analysis.
7. CISA. (2014). GameOver Zeus P2P Malware Alert (TA14-150A). US Cybersecurity and Infrastructure Security Agency.
8. Wang, X., & Zhao, Y. (2014). Protecting Financial Institutions from Man-in-the-Browser Attacks. *Virus Bulletin Conference*.
9. Sood, A. K., & Enbody, R. J. (2014). Evolution of Domain Generation Algorithms. *SecureWorks Research*.
10. Plohmann, D., & Gerhards-Padilla, E. (2012). Case Study of the Miner Botnet. In *2012 7th International Conference on Malicious and Unwanted Software (MALWARE)*, 78-86.